

鹰眼长空 — 开发文档

面向新开发者的完整上手指南。

1. 项目概述

双模块无人机轨迹监测系统：

模块	端口	职责
trajectory_reconstruction	5000	轨迹加载、3D 可视化、预测、AI 策略、分析
trajectory_recognition	5001	轨迹模式识别与分类（框架阶段）

技术栈：Python Flask · pywebview · Three.js · ECharts · OpenAI 兼容 AI 接口

2. 环境搭建

```
pip install flask pywebview requests
```

配置 config.json

```
{
  "ai": { "api_key": "...", "url": "...", "model": "..." },
  "detection_methods": {
    "visible": { "name": " ", "color": "#ff6b6b", "weight": 1.0 },
    "infrared": { "name": " ", "color": "#4ecdc4", "weight": 1.0 },
    "radar": { "name": " ", "color": "#ffe66d", "weight": 1.0 },
    "self": { "name": " ", "color": "#FF9500", "weight": 1.0 }
  },
  "prediction_settings": { "min_points": 10, "max_points": 50, "default_points": 20, "time_step": 0.3 },
  "capture_weights": { "height": 0.3, "speed": 0.3, "acceleration": 0.2, "curvature": 0.2 },
  "camera_speed": 0.12,
  "theme": "dark"
}
```

兼容任意 OpenAI Chat Completions 接口：

```
{ "ai": { "url": "https://api.deepseek.com/v1/chat/completions", "model": "deepseek-chat" } }
{ "ai": { "url": "http://localhost:11434/v1/chat/completions", "model": "llama3" } }
```

启动

```
python main.py           #
python main.py --headless # HTTP
python main.py recon      # → :5000
```

3. 目录结构

```
drone/
  main.py          #
  config.json

trajectory_reconstruction/
  app.py           # Flask
  core/
    state.py       # detection_methods
    config/config_manager.py # / /
    io/data_loader.py # .dat I/O
    prediction/prediction.py #
    ai/ai_service.py # API
  services/
    data_service.py # / /
    predict_service.py #
    backup_service.py #
  views/
    main.py        # /docs, /settings
    api.py         # & API
    api_predict.py # & AI API
    api_report.py  #
    analysis.py    # +
  frontend/
    pages/         # Jinja2 icons.html, docs.html, settings.html
    static/
      css/main.css # GitHub
      js/
        lib/       # Three.js + ECharts
        common/
          three-utils.js # 3D / / /
          three-input.js # WASD/ /resize
          toast.js       # Toast
          utils.js       # lerp
          pages/         # JS

trajectory_recognition/ #
data/                   # fact/predict/backup
reports/
templates/             # config_template.json
docs/
```

4. 分层架构

```
frontend/  ←
views/     ← HTTP + JSON
services/  ←
core/      ←
```

依赖方向: views → services → core (单向)

5. 核心模块详解

5.1 core/state.py — 全局状态

```
detection_methods = {
    "visible": {"name": " ", "color": "#ff6b6b", "weight": 1.0,
               "visible": True, "points": [...], "timestamps": [...]},
    "infrared": {...}, "radar": {...}, "self": {...}, "synthetic": {...}
}
```

生命周期: `init_from_config()` → `initialize_data()` → API 增删改 → `save_metadata()` 持久化到 `config.json`。

5.2 core/config/config_manager.py

`ensure_config()` 确保配置文件存在, `_validate_config()` 自动补全缺失字段 (包括 `weight`、`capture_weights`、`camera` 等)。

5.3 core/io/data_loader.py

```
load_dat_file(path)           → (points, timestamps)
save_predict_data(id, pts, ts) #   data/predict/
load_default_data()           #   fact1-3.dat
```

5.4 core/prediction/prediction.py

最后两点速度向量线性外推。首个预测点为历史末点 (保证连接), 共返回 `num_points+1` 个点。

5.5 前端 JS 架构

文件	职责
<code>three-utils.js</code>	<code>buildAxes</code> / <code>buildLights</code> / <code>buildStarfield</code> / <code>getSceneBackground</code> (双主题)
<code>three-input.js</code>	<code>bindWASD</code> / <code>bindCoordTooltip</code> / <code>bindResize</code>
<code>icons.html</code>	25 个 SVG 图标 Jinja2 宏
<code>dashboard.js</code>	3D 场景/轨迹/动画/平台切换
<code>predict.js</code>	预测 + 3D 预览/动画
<code>analysis.html JS</code>	ECharts 图表 + 捕捉时机卡片
<code>data.js</code>	上传/备份/恢复/更新

6. 数据流

启动

```
main.py → app.create_app()
        → ensure_config() → init_from_config() → initialize_data()
        → fact/*.dat →      →
```

预测

```
→ POST /api/predict_all
→ predict_service.predict_all()
→ predict_single() → generate_prediction()
→ = _synthesize_predictions
→ data/predict/pre*.dat
```

综合轨迹合成

```
synthesize_trajectory()
→ weight
→
→ +
→ _nearest_avg
→
```

备份恢复

```
: create_backup() → fact/predict + dump memory → manifest.json
: restore_backup() → _clear_all() → → → synthesize_trajectory()
```

7. API 参考 (18 个端点)

数据

端点	说明
POST /api/refresh_data	重载出厂数据
POST /api/load_data	上传自选 (multipart)
POST /api/clear_all_data	备份后清空
POST /api/toggle_method	切换平台可见性

预测 & AI

端点	说明
POST /api/predict	单平台预测
POST /api/predict_all	全平台预测
POST /api/ai_suggestion	AI 策略
POST /api/save_report	保存报告

备份

端点	说明
GET /api/list_backups	列出备份
POST /api/restore_backup	恢复指定
POST /api/restore_all_backups	一键恢复
POST /api/backup/create	创建备份
POST /api/backup/delete	删除备份

端点	说明
----	----

分析

端点	说明
GET /analysis/data	运动学数据
GET /analysis/capture	捕捉时机

其他

端点	说明
GET/POST /api/theme	主题读写
POST /api/synthesize	手动合成

8. 添加新功能指南

新 API

在对应 views 文件添加路由，业务逻辑放 services，算法放 core。

新页面

1. frontend/pages/new.html → {% extends "base.html" %}
2. frontend/static/js/pages/new.js → ES6 模块
3. views/main.py → @main_bp.route('/new')
4. base.html 导航栏添加链接

新平台

1. config.json detection_methods 添加条目
2. data/fact/ 放入 .dat 文件
3. data_loader.py 添加文件映射
4. _METHOD_ORDER 添加新 key

9. 代码规范

- **Python:** 类型注解，core 不导入 Flask，单向依赖
- **JS:** ES6 模块，async/await，共享逻辑提取到 common/
- **CSS:** 所有值用 var(--xxx)，支持 [data-theme="light"]
- **提交:** 中文描述，简洁准确